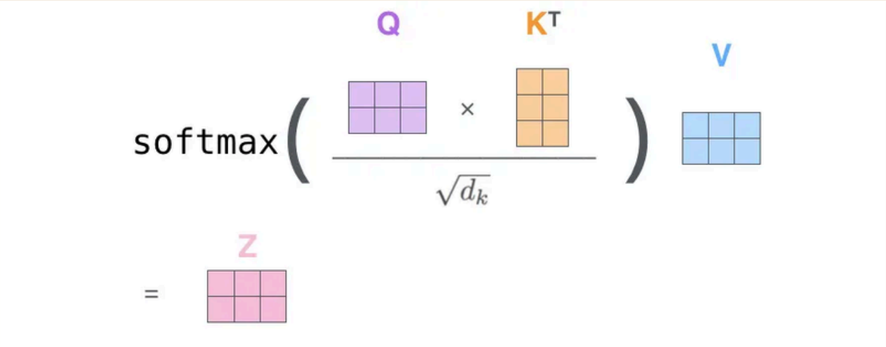
CS6886 - PROJECT PROPOSAL

Optimising KV caching in GQA using custom compiler passes

By Nishanth S (EE23B049) and Dhruv Prasad (EE23B130)

Self/Single Head Attention

- Sentence - tokenization - vector embeddings.
- Want to find out good next token given current tokens.
- K,Q,V, 3 different representations of the current token. Formed from input and weight



- Output matrix gets context of the entire set of input tokens, help predict next token

Multi - Head Attention

- One attention head not sufficient to capture all the different types of relationships.
- K,Q,V matrices are split into smaller equal sized chunks, sent to individual heads.
- Individual heads outputs are concatenated.
- Each head does computation independantly, good to write cuda code for.
- Also helps attention heads capture relationships (rephrase)

How does inference happen?

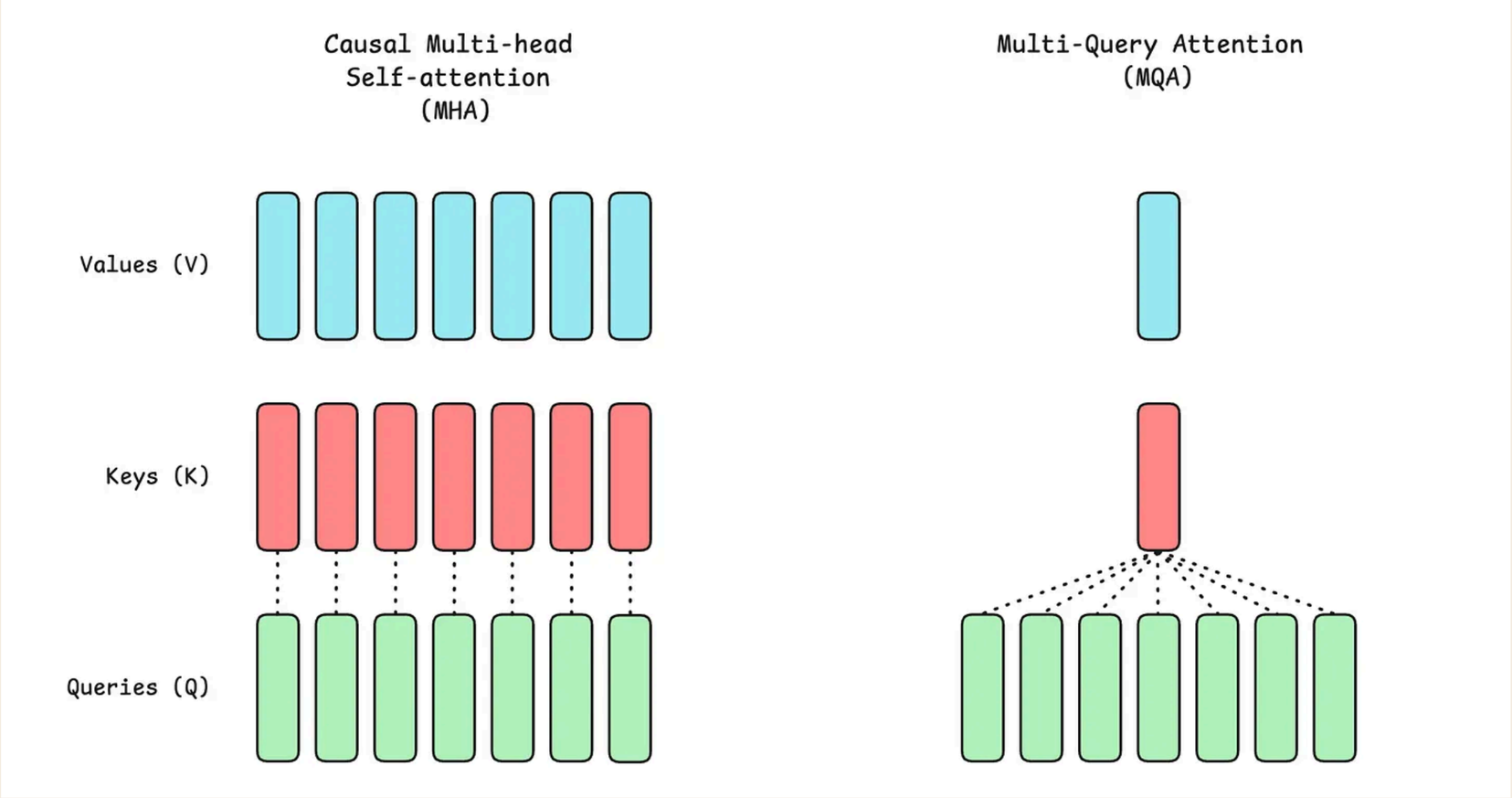
- Lets say the prompt is ‘Does zaitoon serve good veg food?’ Model is trained, weights fixed.
- Input is tokenized, embedded, and represented using K,Q,V using weights.
- Ouput after attention layers is a probability distribution for next token- choose next token.
- Lets say ‘No’ is the next best token predicted by the model.
- New input is ‘Does zaitoon serve good veg food? No’ and sent through the same cycle.

KV Caching

- We dont want to recompute K and V for every new token- wasted computation
- Cache the K and V rows for the old tokens and store them.
- Unfortunately, this might be too much data for us to store.
- Computing K,V from scratch for every new token is too slow, inference should be fast

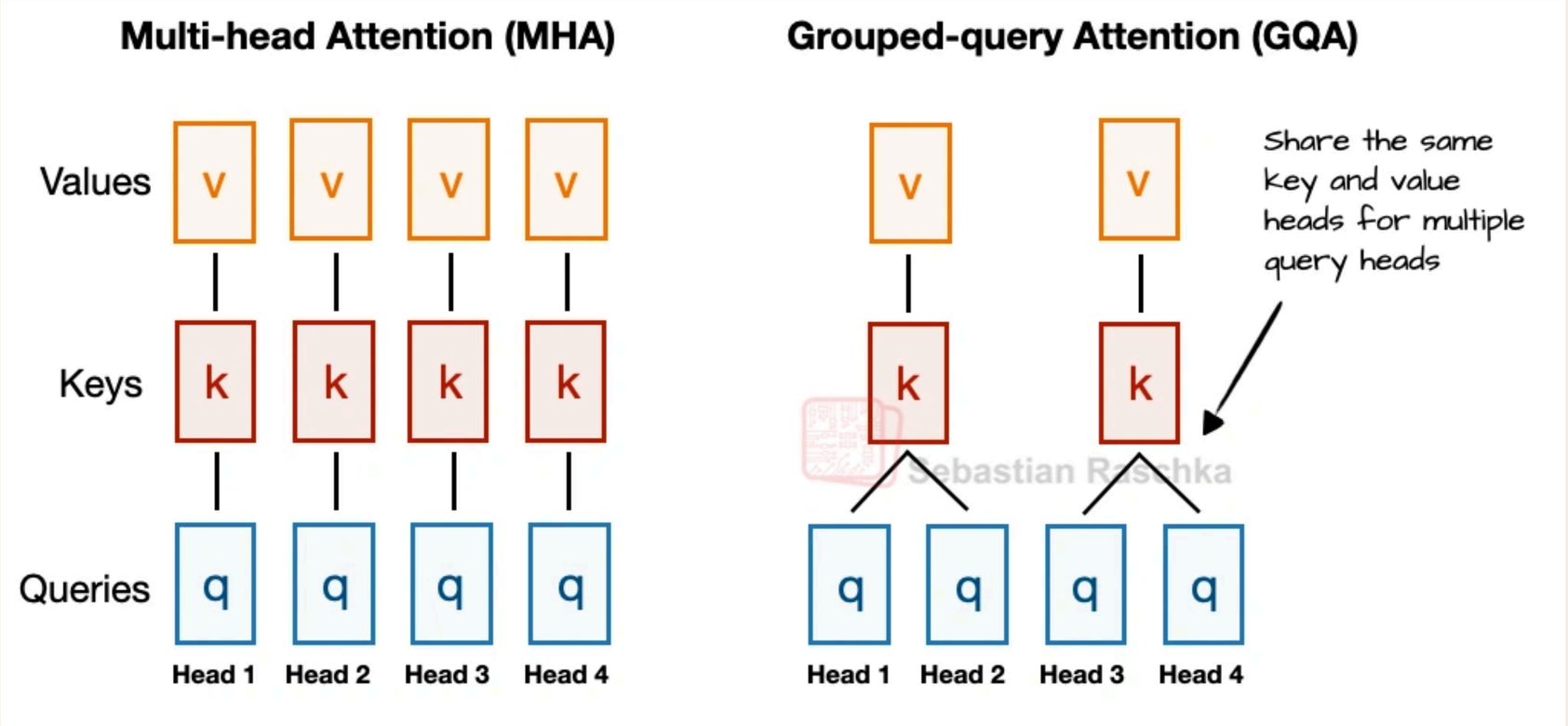
Multi Query Attention

- A proposed way to decrease inference time
- Smart people decided use the same K and V in all the heads, so instead of h different K and V, we have one for all the heads, which get appended for every new token.
- Inference time is much better(since easier to cache), but accuracy takes a hit.



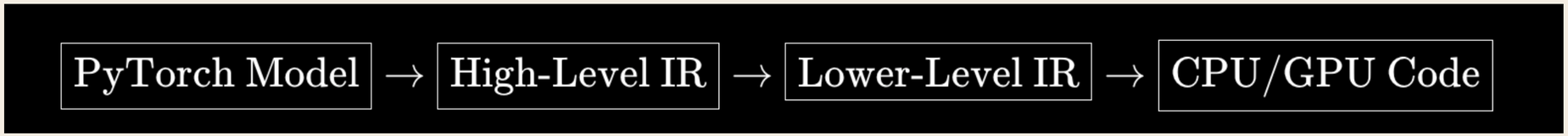
Group Query Attention

- Can be understood as a tunable form of MQA.
- We choose the number of different (K,V) matrices that is used across all heads.
- So we can have 2 K,V matrices shared across 8 heads (one set of matrices for 4 heads.
- Accuracy Vs Memory usage can be tuned as per requirement



MLIR/IREE Compiler

- IREE (Intermediate Representation Execution Environment) is an open-source compiler/runtime for deploying ML models efficiently across hardware such as CPUs and GPUs.
- The code goes through multiple ‘intermediate representations’ before finally becoming code that runs on CPU/GPU
- At different stages, the compiler can apply optimizations and transformations appropriate to that level of abstraction



Issue we found

- Lets say we use 2 K and V matrices in our GQA.
- Ideally you would want the heads to use these matrices directly.
- GQA shares K/V heads, but PyTorch/initial IR steps expresses this by logically expanding K/V to match the number of query heads.
- This expansion survives IREE lowering and appears as a separate expanded K/V intermediate buffer.
- Opportunity: avoid materializing this intermediate and directly reuse the original shared K/V heads during attention.

Proof

- We wrote a GQA and looked at the intermediate represensations
- KV head 0 → K0 K0 K0 K0
- KV head 1 → K1 K1 K1 K1
- We verified that the data going to the heads are of this form
- This introduces redundant K/V storage and memory traffic, partially negating GQA's intended memory-efficiency benefit.

```
# GQA: 8 query heads, 2 KV heads
group_size = query_heads // kv_heads   # 4

key   = key.repeat_interleave(group_size, dim=1)
value = value.repeat_interleave(group_size, dim=1)

scores = torch.matmul(query, key.transpose(-2, -1))
output = torch.matmul(softmax(scores), value)
```

```
%expanded = tensor.empty()
    : tensor<1x2x4x16x32xf32>

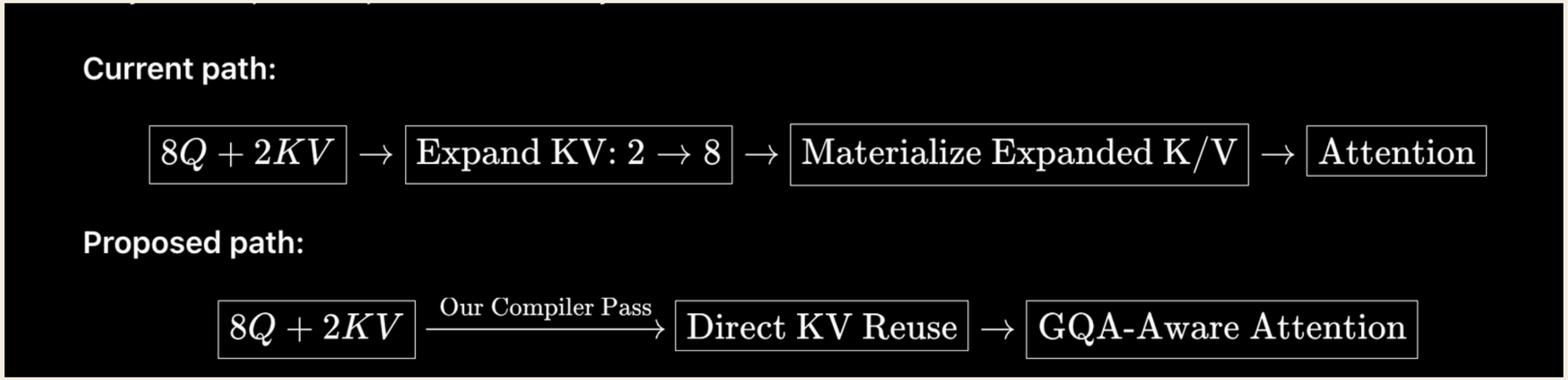
%broadcast = linalg.generic
    ins(%K : tensor<1x2x16x32xf32>)
    outs(%expanded : tensor<1x2x4x16x32xf32>) {
  ^bb0(%k: f32, %out: f32):
    linalg.yield %k : f32
}

%K8 = tensor.collapse_shape %broadcast
    : tensor<1x2x4x16x32xf32>
      into tensor<1x8x16x32xf32>
```

Things we can do to fix it

- Detect the GQA pattern during IREE lowering.
- Remove the expanded K/V intermediate created before attention.
- Map each query head directly to its shared KV head, preserving GQA's intended KV reuse.
- Expected benefit: lower memory traffic and faster attention execution.
- We can do this by storing the K/V in shared memory of GPU, or enforce caching in CPUs

- Our project, make a compiler pass to do this.



Prior work done in this field

- From the literature survey we did, as well as making LLM scan the internet for similar ideas, we havent found any work on the exact same idea.
- Some related work:
- FlashInfer - They do send precomputed K and V rows that are reused into shared memory, but its something manually done, not something automatically done by a compiler. Also it proves that sending these vectors into shared memory does result in speedup.

What we plan to achieve

- We want to write this compiler pass, and truly see if it gives some improvement.
- Check the assembly for different targets to see if it makes a difference there
- Once verified, we will benchmark it with the present IREE to see if there is any speedup in GQA.
- We also will benchmark against the number of memory accesses that are not cached.
- Other similar compiler issues that we may find upon progressing in the project.
- Other benchmarks which we will think of when we get there.